RepLeafGBM: Gradient Boosting with Raw-Feature Routing and Representation-Conditioned Leaves

Masaya Kawamata

June 29, 2026

Abstract

RepLeafGBM is a gradient-boosted tree ensemble whose trees *route* on raw, interpretable features while each *leaf* predicts through a learned, frozen representation $z_\theta(x)$. It occupies the space between classic gradient-boosted decision trees (GBDTs), whose leaves emit a constant, and tabular deep learning, which embeds features but discards the axis-aligned tree. Routing stays axis-aligned, categorical-native, and readable; expressivity is added only inside the leaf, as an h-weighted ridge fit of an affine model on $z_\theta(x)$. We give the additive/Newton formulation, the constant and embedded-linear leaf models, and a supervised *pretrain-then-freeze* encoder, together with a proposition showing that updating the encoder during boosting breaks the stage-wise additive structure of gradient boosting—which is why the encoder is frozen. We then study the method empirically and report an honest picture. Under a *shared* hyperparameter-optimization budget on the Grinsztajn tabular benchmarks, with significance testing, RepLeafGBM is a competitive *mid-pack* learner—not state-of-the-art on average. Its distinctive value is in niche regimes: the representation-conditioned leaf is a genuine, separable contribution (refitting an external GBDT's own routes with embedded-linear leaves improves regression by up to -11.5% RMSE), and joint vector-leaf routing with robust objectives is decisive under target contamination—where we identify and fix a scale bug in saturated-gradient (Huber/quantile) objectives through per-output target standardization, making them robust *consistently* across target scales. Equally, we document where the idea does *not* help—binary classification (the logistic Hessian starves the leaf-linear fit), and *learned* encoders, which beat `identity` on structured synthetic data but fail to transfer to real tabular targets. We release an open-source implementation with parity-tested NumPy, Rust ($\sim$5.8$\times$), and CUDA (up to 5.5$\times$) backends.

1 Introduction

Gradient-boosted decision trees (GBDTs) remain the dominant model class on tabular data [3, 6, 13, 19], and recent benchmarks confirm that they still match or outperform deep tabular models on typical problems [10]. Their strength is the axis-aligned split: routing on raw features handles discontinuities, interactions, missing values, and categorical columns cheaply and interpretably. Their weakness is the *leaf*: a classic GBDT emits a constant per leaf, so a smooth within-region effect—a price gradient, a redshift trend, a dose response—can only be approximated by stacking many small steps.

Tabular deep learning attacks the opposite end. Numerical-feature embeddings, notably the piecewise-linear and periodic families of Gorishniy et al. [9], let neural models capture smooth nonlinear structure that constant-leaf trees represent only coarsely [2, 8, 18]. But these models embed features as the *input* to a deep network and discard the axis-aligned tree, giving up the routing that makes GBDTs robust on messy tables.

RepLeafGBM is a deliberately *asymmetric* hybrid that keeps both halves. Trees route on raw features exactly as in a histogram GBDT; each leaf, instead of a constant, holds a small ridge-regularized affine model over a learned representation $z_\theta(x)$. The embedding does the smooth interpolation *inside* a region, while the representation is never used for splitting—so routing stays axis-aligned, categorical-native, and readable, and the search never suffers an embedding-induced dimensionality blow-up. Crucially, the encoder is fitted once before boosting and then *frozen*; we prove (Proposition 1) that updating it mid-boosting would retroactively change every earlier tree's output and break the stage-wise additive interpretation of boosting. RepLeafGBM is therefore neither a neural network placed inside a tree nor a tree grown over embeddings.

Contributions.

1. **The RepLeafGBM architecture** (§4–5): raw-feature routing with representation-conditioned constant and embedded-linear leaves, fitted in the Newton / h-weighted-least-squares GBDT framework, with a prediction-time extrapolation guard and a constant-leaf fallback.
2. **A correctness argument for freezing** (§6): a proposition that a mid-boosting encoder update breaks stage-wise additivity, motivating the pretrain-then-freeze design and delimiting what alternating optimization would require.
3. **A supervised pretrain-then-freeze encoder** (§7) whose target is the negative gradient at the initial score, with an analysis of why $-g$ (not the Newton target $-g/h$) is the right pretraining signal.
4. **Multiclass and multi-output regimes, and scale-consistent robust objectives** (§8, §9.5): one-tree-per-class softmax and a shared-routing vector leaf that solves K ridge problems from one Gram factorization, including robust (Huber/quantile) objectives. We identify that their saturated gradients under-fit large-scale targets and fix it with per-output target standardization, turning robust multi-output regression into a *general* win under contamination.
5. **An honest empirical study** (§9): a fair, same-HPO-budget leaderboard on the Grinsztajn tabular suites with significance testing (RepLeafGBM is competitive mid-pack), niche studies (leaf-channel isolation via router extraction; robust multi-output under contamination), and an ensemble-diversity probe. We report null results as prominently as wins.
6. **An open-source implementation** with parity-tested NumPy, Rust, and CUDA split/leaf backends.

2 Related work

Gradient-boosted decision trees. Gradient boosting [6] and its second-order, regularized formulations [3, 5] underlie modern histogram GBDTs. LightGBM [13] contributes leaf-wise growth and gradient-based categorical subset splits; CatBoost [19] uses oblivious (symmetric) trees as a strong regularizer and fast predictor. RepLeafGBM reuses this routing machinery verbatim—histogram splits, Newton gain, leaf-wise/depthwise/ symmetric policies, categorical subset splits—and changes only the leaf.

Model trees and piecewise-linear leaves. Replacing the constant leaf with a local linear model is a long line of work: model trees [20, 23], logistic model trees [16], and, within boosting, gradient boosting with piecewise-linear regression trees [21]. These fit a linear model in the *raw* (or selected) features. RepLeafGBM's leaf instead regresses on a *learned, frozen* representation $z_\theta(x)$, of which the raw-linear leaf is the identity-encoder special case.

Tabular deep learning and numerical embeddings. Neural tabular models embed features and learn interactions end-to-end: TabNet [2], NODE [18], and FT-Transformer [8]. Gorishniy et al. [9] show that piecewise-linear (PLR) and periodic numerical embeddings are the key ingredient; these are exactly the encoder families RepLeafGBM can consume. The difference is how the embedding is used: as the input to a deep network there, versus a frozen *leaf basis* under an explicit tree here. Grinsztajn et al. [10] document that tree-based models still lead on typical tabular data, which motivates keeping the tree explicit.

Hybrids of trees and representations. Deep neural decision forests [15] and NODE [18] make routing itself soft/differentiable over learned features; DeepGBM [14] distills GBDT structure into a network. All of these let representations influence *routing*. RepLeafGBM deliberately does the opposite: routing is a hard, raw-feature decision tree, and the representation acts only as the regressor basis inside each leaf—preserving GBDT routing semantics while adding leaf expressivity.

3 Notation and gradient boosting

We are given n examples $\{(x_i, y_i)\}_{i=1}^n$ with raw feature vectors $x_i \in \mathbb{R}^d$ (numeric and categorical entries) and targets y_i. A differentiable loss $L(y, F)$ measures the discrepancy between a target and a real-valued raw score F (for K-way problems, $F \in \mathbb{R}^K$). Gradient boosting [6] builds an additive model stage-wise,

$$F_0(x) = \arg\min_c \sum_{i=1}^n L(y_i, c), \qquad F_t(x) = F_{t-1}(x) + \eta\, f_t(x), \tag{1}$$

where $\eta \in (0, 1]$ is the learning rate and f_t is the round-t weak learner. At round t we expand L to second order around the current score using the per-row gradient and Hessian

$$g_i = \left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{t-1}(x_i)}, \qquad h_i = \left.\frac{\partial^2 L(y_i, F)}{\partial F^2}\right|_{F=F_{t-1}(x_i)}. \tag{2}$$

Dropping constants, the round-t objective is the quadratic

$$\sum_i \Big[\, g_i f(x_i) + \tfrac{1}{2} h_i f(x_i)^2 \,\Big] + \Omega(f) = \tfrac{1}{2} \sum_i h_i \big(f(x_i) - t_i\big)^2 + \text{const} + \Omega(f), \qquad t_i = -\frac{g_i}{h_i}, \tag{3}$$

where $\Omega(f)$ is an L_2 regularizer on the leaf parameters. Equation (3) is the workhorse identity: *every* weak learner, whatever its leaf model, is fitted by **h-weighted least squares on the Newton targets** $t_i = -g_i/h_i$. For squared error ($h_i \equiv 1$, t_i the residual) this is exact rather than approximate.

Sample weights. Per-row weights $\omega_i \geq 0$ scale each loss term, so g_i, h_i are replaced by $\omega_i g_i, \omega_i h_i$ everywhere. The Newton target $t_i = -g_i/h_i$ is unchanged but its fitting weight becomes $\omega_i h_i$; folding the weight into (g, h) before any histogram is built leaves the split and leaf kernels untouched.

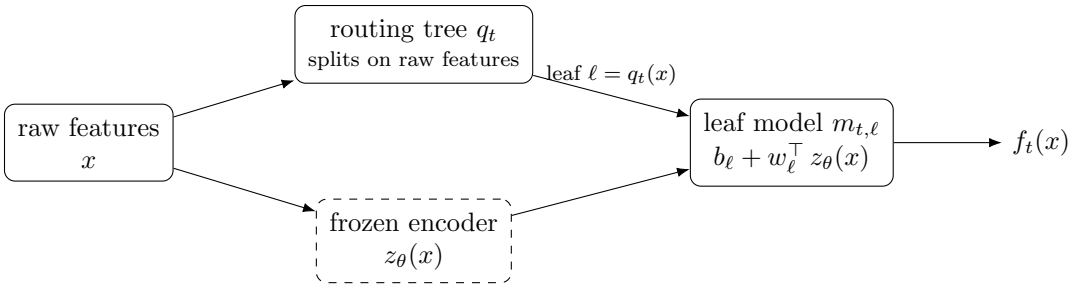

Figure 1: A RepLeafGBM round-t weak learner. The tree q_t routes x to a leaf using *raw features only* (invariant I1); the *frozen* encoder produces $z_\theta(x)$ (invariant I2); the selected leaf ℓ predicts the affine model $b_\ell + w_\ell^\top z_\theta(x)$. The boosted model sums $F_T(x) = F_0 + \eta \sum_t f_t(x)$. The representation never enters a split, and the encoder is fixed throughout boosting.

4 RepLeafGBM: raw routing and representation-conditioned leaves

A classic GBDT weak learner is a regression tree whose leaves emit a constant. RepLeafGBM keeps the boosting and routing machinery but upgrades the leaf. Let $z_\theta : \mathbb{R}^d \to \mathbb{R}^{d_z}$ be an encoder producing a learned representation, fitted once and then *frozen* (Section 6). A round-t weak learner is a pair $(q_t, \{m_{t,\ell}\})$:

- a **routing tree** $q_t : \mathbb{R}^d \to \{1, \dots, L_t\}$ that maps an example to a leaf index using *raw features only* — the representation never enters a split test; and
- a per-leaf **leaf model** $m_{t,\ell}$ evaluated on the representation (Figure 1), so

$$f_t(x) = m_{t,\, q_t(x)}\big(z_\theta(x)\big). \tag{4}$$

With the affine (*embedded-linear*) leaf of Section 5, the full ensemble after T rounds is

$$F_T(x) = F_0 + \eta \sum_{t=1}^{T} \Big[\, b_{t,\, \ell_t(x)} + w_{t,\, \ell_t(x)}^\top z_\theta(x) \,\Big], \qquad \ell_t(x) = q_t(x). \tag{5}$$

Two invariants define the method and are preserved throughout:

(I1) Raw-feature routing. Splits are chosen from raw features only; the tree partition is axis-aligned and human-readable, exactly as in LightGBM/XGBoost [3, 13], and categorical columns route through native subset splits. The embedding dimensions are never split on.

(I2) Frozen encoder. z_θ is fitted before boosting and held fixed; past leaves' parameters depend on z_θ, so updating θ mid-boosting would break the stage-wise additive assumption (Section 6).

RepLeafGBM is therefore *not* a neural network placed inside a tree, and *not* a tree grown over embeddings: routing remains a raw-feature decision tree, and the learned representation acts only as the regressor basis inside each leaf.

Routing: split selection. Trees are grown greedily on histograms of the raw features, scoring a candidate split of a node with row set I into children I_L, I_R by the standard Newton gain [3]

$$\text{gain} = \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda}, \qquad G_{(\cdot)} = \sum_{i \in I_{(\cdot)}} g_i, \quad H_{(\cdot)} = \sum_{i \in I_{(\cdot)}} h_i, \tag{6}$$

with $\lambda =$ `l2_leaf`, `min_samples_leaf` enforced on both children, and missing values routed to the left child. The growth policy (`leafwise` [13] best-gain-first by default; also `depthwise` and oblivious `symmetric` [19]) only changes *which* node is expanded next, not the per-split score; (I1) holds for all policies.

5 Leaf models

Given a fixed partition, each leaf is fitted independently by the h-weighted least squares of (3) restricted to its rows I_ℓ.

Constant leaf. The minimizer of $\sum_{i\in I_\ell} h_i(b - t_i)^2 + \lambda b^2$ is the XGBoost-style damped Newton step

$$b_\ell = -\frac{\sum_{i\in I_\ell} g_i}{\sum_{i\in I_\ell} h_i + \lambda}. \tag{7}$$

Embedded-linear leaf. Let $z_i = z_\theta(x_i)$. The leaf fits an affine model on the representation by ridge regression [11] with an *unpenalized* intercept,

$$\min_{b,\,w} \sum_{i\in I_\ell} h_i\big(b + w^\top z_i - t_i\big)^2 \;+\; \lambda\|w\|^2. \tag{8}$$

Centering z and t by their h-weighted means $\bar z, \bar t$ over I_ℓ decouples the intercept and yields the ridge normal equations

$$\big(Z_c^\top H Z_c + \lambda I\big)\, w = Z_c^\top H\, t_c, \qquad b = \bar t - w^\top \bar z, \tag{9}$$

where Z_c, t_c are the centered design and targets and $H = \mathrm{diag}(h_i)$. Equation (7) is the special case $w = 0$. The *raw-linear* variant replaces z_i with standardized raw numeric features.

Extrapolation guard. Each linear leaf stores the per-dimension range $[z_{\min}, z_{\max}]$ of the embeddings it was fitted on, and predicts with $\mathrm{clip}(z, z_{\min}, z_{\max})$: the leaf's response is its fitted affine function inside the training support and constant outside it. Training rows lie inside by construction, so the training trajectory is unchanged; only out-of-support queries are clamped.

Conditioning and fallback. The system (9) can be ill-conditioned for small or locally degenerate leaves. The leaf falls back to the constant model (7) when $|I_\ell| < \max(2\,$`min_samples_leaf`$, d_z + 2)$, or when the solve fails or returns non-finite weights; $\lambda > 0$ is required in practice.

6 The frozen-encoder design and why it is necessary

Invariant (I2) is not a convenience — it is required for the boosting objective to remain well defined. Consider the ensemble (5). Each leaf weight $w_{t,\ell}$ was fitted against the value of z_θ *as it stood at round* t.

Proposition 1 (Encoder updates break stage-wise additivity)**.** *Suppose the encoder parameters are changed from θ to θ' at some round $T' > t$, after trees $1, \dots, t$ have been fitted. Then*

1. *the outputs of all earlier trees $1, \dots, T'$ change retroactively, since each emits $b_{\cdot,\ell} + w_{\cdot,\ell}^\top z_{\theta'}(x) \neq b_{\cdot,\ell} + w_{\cdot,\ell}^\top z_\theta(x)$;*

2. *consequently the cached raw scores F used to form (g, h) no longer equal the model's actual output, so every subsequent gradient is evaluated at the wrong point; and*

3. *the interpretation of boosting as coordinate descent in function space — each stage greedily fitting the current residual — no longer holds.*

Restoring correctness after an encoder update would require re-evaluating (or re-fitting) all earlier leaf weights and invalidating the prediction cache, i.e. alternating optimization between θ and the ensemble. RepLeafGBM v0 instead adopts the simplest sound choice: *fit z_θ once before boosting, then freeze it.* The representation is thus a fixed, supervised feature map, and the leaf solves (9) are ordinary ridge regressions against a constant basis.

Remark 1. *Freezing also keeps the leaf fit convex and closed-form, makes training embarrassingly reusable across rounds (the embedding $Z = z_\theta(X)$ is computed once), and preserves a clean separation of concerns: the encoder owns the representation, the booster owns gradients, tree growth, and leaf fitting.*

7 Supervised encoder pretraining

A *learned* encoder is fitted once, before boosting, on a supervised target and then frozen. The target is the **negative gradient of the loss at the constant initial score** F_0 — precisely the residual the first tree would chase:

$$\begin{aligned}
\text{regression:} \quad & t = -g = y - \overline{y}, && h = 1, \\
\text{binary:} \quad & t = -g = y - \sigma(F_0), && \\
\text{multiclass:} \quad & T = -g = \text{onehot}(y) - \text{softmax}(F_0) \in \mathbb{R}^{n \times K}, && F_{0,k} = \log \text{prior}_k, \\
\text{multi-output:} \quad & T = -g = Y - \overline{Y} \in \mathbb{R}^{n \times K}, && F_{0,k} = \overline{Y}_{\cdot k}.
\end{aligned} \tag{10}$$

A scalar target trains a Linear$(d_z, 1)$ head on $z_\theta(x)$; an (n, K) matrix target trains a Linear(d_z, K) head, with the mean-squared error averaged over rows and outputs so the representation is pushed to be linearly predictive of every output residual at once. Each output column is standardized to unit variance so no class/output dominates, and the head is discarded after fitting. Two choices are worth recording.

Why $-g$ rather than the Newton target $-g/h$. Pretraining should align the representation with the loss's steepest-descent direction; the Hessian reweighting belongs to the leaf solve, not to feature learning. Moreover, at F_0 the multiclass Hessian $h_k = p_k(1 - p_k)$ is *column-constant* (because softmax(F_0) is the row-independent prior), so $-g/h$ equals $-g$ up to a per-column scale and **coincides with it after per-output standardization**. The two formulations are identical here, and $-g$ keeps scalar and vector targets consistent.

What the encoder learns. At F_0 the multiclass target reduces to $T = \text{onehot}(y) - \text{prior}$, a centered class-membership indicator. Pretraining therefore teaches z_θ a representation from which class membership is *linearly* recoverable — exactly the signal the per-class linear leaves consume.

Encoder families. The leaf basis z_θ is pluggable, and only its output is consumed downstream; the booster is agnostic to how z_θ is produced. We use two groups. *Fixed* encoders require no learning: `identity` standardizes the raw numeric features ($z_j = (x_j - \mu_j)/\sigma_j$) and is the default; `plr` appends

Algorithm 1 RepLeafGBM training

Require: data (X, y); loss L; rounds T; rate η; leaf capacity / policy; ridge λ; encoder family; optional sample weights ω

$F_0 \leftarrow \arg\min_c \sum_i L(y_i, c)$ ▷ (weighted) optimal constant: mean / log-odds / class-prior / quantile
$T_{\text{pre}} \leftarrow -\nabla_F L(y, F_0)$ ▷ pretraining target: residual at F_0 (Section 7)
fit encoder θ on $X \to T_{\text{pre}}$; **freeze** θ
$Z \leftarrow z_\theta(X)$ ▷ precompute frozen embeddings
$F \leftarrow F_0$ **for all rows**
for $t = 1$ **to** T **do**
    $g_i \leftarrow \partial_F L(y_i, F_i), \quad h_i \leftarrow \partial_F^2 L(y_i, F_i)$ ▷ scale by ω_i if weighted
    $t_i \leftarrow -g_i/h_i$ ▷ Newton targets
    $q_t \leftarrow \textsc{GrowTree}(X_{\text{raw}}, g, h, \texttt{leaves}, \texttt{policy})$ ▷ splits on raw features via gain (6)
    for each leaf ℓ with rows I_ℓ **do**
        fit $m_{t,\ell}$ on $\big(Z_{I_\ell},\, t_{I_\ell}\big)$ with weights h_{I_ℓ} ▷ constant (7) or ridge-on-z (9); fallback if ill-conditioned
    end for
    $f_t \leftarrow (q_t, \{m_{t,\ell}\}); \quad F \leftarrow F + \eta\, f_t(X)$
    optional: early-stop on a validation metric
end for
return $\big\{F_0,\, \eta,\, (q_t, \{m_{t,\ell}\})_{t=1}^{T},\, \theta\big\}$

Algorithm 2 RepLeafGBM prediction for a query x

$z \leftarrow z_\theta(x); \quad F \leftarrow F_0$
for $t = 1$ **to** T **do**
    $\ell \leftarrow q_t(x)$ ▷ route on raw features (missing → left)
    $F \leftarrow F + \eta\big(b_{t,\ell} + w_{t,\ell}^{\top} \operatorname{clip}(z, z_{\min}^{t,\ell}, z_{\max}^{t,\ell})\big)$ ▷ $w_{t,\ell} = 0$ for a constant leaf
end for
return $\phi(F)$ ▷ output transform ϕ: identity / σ / softmax / exp

a fixed per-feature piecewise-linear basis over quantile bins; `periodic` appends random sinusoidal features; and `cross` appends a few residual-correlated pairwise products. *Learned* encoders (optional, fitted by the pretraining of this section and then frozen) follow the numerical-embedding families of Gorishniy et al. [9]: `torch_plr` learns a per-feature linear–ReLU projection over the piecewise-linear basis, `torch_periodic` learns the periodic frequencies, and `torch_mlp` adds a small MLP with a raw pass-through for cross-feature structure. Learned encoders import a deep-learning framework only at fit time; transform and serialization are pure NumPy, so a saved model predicts without it. If an encoder's output dimension exceeds a cap `max_leaf_emb_dim`, a fixed Gaussian random projection reduces it, bounding the per-leaf ridge cost; §9 measures the practical consequences of these choices.

8 Algorithm

Algorithm 1 states training; Algorithm 2 states prediction. The encoder pretraining (Section 7) happens once, before the boosting loop, and $Z = z_\theta(X)$ is precomputed because z_θ never changes.

Table 1: Leaf-model channel isolated by router extraction (mean over seeds; RMSE for regression, log-loss for binary; lower is better). Holding LightGBM's routes fixed, embedded-linear leaves improve regression; the native RepLeafGBM is best. "n.s." denotes a difference within one standard deviation.

Dataset (metric)	LightGBM	LightGBM routes + embedded leaf	RepLeafGBM (native)
`piecewise_linear` (RMSE)	0.4864	0.4304 (−11.5%)	**0.4074**
`friedman1` (RMSE)	1.1921	1.1689 (−1.9%)	**1.1282**
`binary_piecewise` (log-loss)	0.2850	0.2842 (n.s.)	**0.2804**

Multi-class and multi-output. Two regimes reuse the scalar machinery above. *Multiclass* (softmax) grows *one tree per class* per round on column k of (g, h), with $g_k = p_k - \mathbf{1}[y = k]$ and $h_k = p_k(1 - p_k)$; each class tree is a scalar RepLeafGBM learner. *Multi-output* regression instead grows *one shared tree per round* that routes all K outputs jointly: the split gain sums the per-output gains (6) over a shared partition, and a linear vector leaf solves K ridge problems (9) that — because the constant-Hessian losses give $h \equiv 1$ — share a single centered Gram factorization $Z_c^\top Z_c + \lambda I$ with K right-hand sides.

9 Experiments

We evaluate RepLeafGBM against the strong GBDT baselines LightGBM [13], XGBoost [3], CatBoost [19], and scikit-learn's histogram gradient boosting [17], all trained on the same ordinal-encoded feature matrix, with multiple seeds, a fixed train/validation/ test split, and early stopping on the validation split. Lower is better throughout (RMSE for regression, log-loss for classification, mean per-output RMSE for multi-output). The protocol asks three questions: **(Q1)** is the representation-conditioned leaf a real contribution, separable from routing? **(Q2)** how does RepLeafGBM compare to tuned GBDTs on real tabular data? **(Q3)** when do *learned* encoders help? All numbers below are produced by the benchmark and experiment harness shipped with the implementation (datasets are anchored by OpenML [22] `data_id`; full per-dataset tables and the reproducibility manifest are in Appendix B).

9.1 The leaf-model channel, isolated (Q1)

The cleanest test of the leaf idea holds *routing fixed* and varies only the leaf. We take a LightGBM model, extract its early-stopped tree routes, and refit the leaves two ways: with LightGBM's own constant leaves, and with RepLeafGBM embedded-linear leaves over $z_\theta(x)$ (router extraction; the leaves are refit by sequential replay so the boosting consistency is preserved). Table 1 shows that on the same routes, swapping constant for embedded-linear leaves improves regression by up to 11.5% RMSE, while a fully native RepLeafGBM (grown with embedded leaves interleaved) does better still with far fewer trees. On binary data the leaf swap is neutral—a recurring theme below.

On real regression data the same isolation holds: refitting an identity-embedded linear leaf on extracted LightGBM routes for `california`, `diamonds`, and `wine_quality` (five seeds) improves RMSE on all three datasets and never loses on a single seed (median ∼1–2%), while a PLR encoder adds nothing over identity—the gain is the linear leaf, not a richer representation, reinforcing `identity` as the default (§9.4). With only five seeds these per-dataset Wilcoxon tests sit at the two-sided $p = 0.0625$ floor, so the direction is consistent but formal significance awaits more seeds.

Table 2: Real-data regression RMSE (↓; log1p target for `house_sales`/ `diamonds`, 15k rows, 2 seeds). The extrapolation guard resolves the `diamonds` blow-up and helps everywhere; categorical subset splits then give `diamonds` the win over LightGBM with native categoricals. `california` has no categorical features.

Dataset	embedded leaf (pre-guard)	+ extrapolation guard	+ categorical subset splits	LightGBM (native cat.)
`california`	0.4743	0.4594	—	0.4628
`house_sales`	0.1681	0.1667	0.1660	**0.1652**
`diamonds`	0.2760	0.0953	**0.0940**	0.0948

9.2 Leaf models and the extrapolation guard on real data

On real regression data, an unguarded embedded-linear leaf can extrapolate catastrophically when a query lands outside a leaf's training support: on `diamonds`, a handful of z-outliers blew the test RMSE up to 0.276. The prediction-time extrapolation guard (§5) clips z to each leaf's fitted range, and recovers a competitive model without changing the training trajectory (Table 2). With the guard—and with native categorical subset splits for low-cardinality categoricals—embedded-linear leaves beat constant leaves on 3/3 real regression datasets and edge out LightGBM with native categorical handling on `diamonds`. High-cardinality categoricals (e.g. `adult`, up to 41 categories) remain a place where LightGBM's categorical pipeline is ahead by $\leq 0.7\%$.

9.3 Real-data leaderboard under a shared HPO budget (Q2)

A credible comparison must tune *every* model under the same budget on a recognized suite. We evaluate on the four Grinsztajn et al. [10] tabular benchmarks—numerical and categorical × regression and classification (OpenML study identifiers 336/337/335/334; 19/16/17/7 datasets)—under a uniform protocol: each model family (RepLeafGBM, LightGBM, XGBoost, CatBoost, scikit-learn HistGB) is tuned by Optuna TPE [1] with an *identical* number of trials (50 trials × 5 seeds), a 70/15/15 split, and a 10k-row training cap; there is no tuned-versus-default asymmetry. We report mean ranks with Friedman tests, Nemenyi critical-difference diagrams, Wilcoxon signed-rank tests, and bootstrap confidence intervals [4]; a result is highlighted only when it is both statistically significant ($p < \alpha$) and exceeds a minimum-relevant-difference band. To keep the comparison symmetric, *all* models—including RepLeafGBM—are fit on the same ordinal-encoded matrix, so RepLeafGBM's native categorical subset splits are disabled here: categorical performance is therefore fair, but native-categorical handling is left untested.

The honest summary (Table 3) is that RepLeafGBM is a *consistent mid-pack* learner. Its mean rank is 3.18–3.43 out of five in *every* suite; it is never the leader, and it is never both significantly *and* practically worse than the best baseline, nor significantly better. The categorical suites are statistically flat (Friedman $p = 0.375$ and 0.788; the categorical-classification suite has only seven datasets and correspondingly low power). RepLeafGBM is thus competitive but not state-of-the-art on average—a useful, honestly-positioned tabular learner whose distinctive value lies in the niche regimes studied below, not in the aggregate leaderboard. Full per-dataset tables and the critical-difference diagrams are produced by the shipped leaderboard harness (Appendix B).

Table 3: Grinsztajn suites under a shared HPO budget: RepLeafGBM mean rank (↓; among five tuned model families, 5 seeds). RepLeafGBM is mid-pack in every suite, with no significant-and-practical win or loss against the best baseline; the categorical suites are statistically flat. The Friedman p is over all five models.

Grinsztajn suite (# datasets)	RepLeafGBM mean rank (of 5)	Friedman p
Numerical regression (19)	3.2–3.4	significant (< 0.05)
Numerical classification (16)	3.2–3.4	significant (< 0.05)
Categorical regression (17)	3.2–3.4	0.375 (flat)
Categorical classification (7)	3.2–3.4	0.788 (flat)

Table 4: Encoder comparison with embedded-linear leaves (RMSE, ↓). Learned encoders win on structured synthetic data but do not transfer to real tabular targets; `identity` (the default) is best on all real datasets and on the interaction-heavy `friedman1`.

Dataset	`identity`	`torch_periodic`	`torch_plr`
`periodic_mix` (synthetic)	0.3933	**0.3405**	0.3871
`friedman1` (synthetic)	**1.1282**	1.2395	1.1655
`california` (real)	**0.4594**	0.4748	0.4628
`house_sales` (real)	**0.1660**	0.1758	0.1682
`diamonds` (real)	**0.0940**	0.0986	0.0953

9.4 Encoders: synthetic structure vs. real transfer (Q3)

Learned encoders behave as *specialists*, not as a better default (Table 4). On synthetic targets with planted smooth/oscillatory per-feature structure (`periodic_mix`), the learned `torch_periodic` encoder beats `identity` by 13%; on a target dominated by a feature product (`interaction_mix`, not shown), interaction-aware encoders (`cross`, 0.460; `torch_mlp`, 0.537) beat `identity` (0.615). But on every real dataset—and on `friedman1`, whose $\sin(\pi x_1 x_2)$ interaction no per-feature encoder can represent—`identity` is best. Adding validation-based early stopping and weight decay to the pretraining loop does not change this picture. The reading is that the raw-feature router already captures whatever structure these encoders extract from real targets, so the extra capacity only adds variance; `identity` is the right default and learned encoders are tools for targets with *known* structure that trees route poorly.

9.5 Multi-output regression and robust objectives

A single shared routing tree with vector leaves lets all outputs share one Gram factorization (§8). On clean data (Table 5) it is decisively best on synthetic multi-output targets and beats LightGBM/HistGB on the real energy-efficiency task, though tuned per-output CatBoost/XGBoost still lead there.

Robust objectives under contamination. The constant-Hessian robust objectives are RepLeafGBM's clearest empirical edge. We inject heavy-tailed outliers (at 8σ) into a fraction $\{0, 4, 8, 16\%\}$ of the training targets and score on clean test data, across five real multi-target datasets—energy-efficiency, jura, water-quality, rf1, and scm20d—and a synthetic control, with squared, Huber [12], and median (quantile, τ=0.5) objectives over five seeds. On energy, jura, water-

Table 5: Multi-output regression, clean leaderboard: mean per-output RMSE (↓, 5 seeds). RepLeafGBM uses one shared routing tree with vector leaves; the baselines fit one model per output.

Setting	RepLeafGBM	CatBoost	XGBoost	LightGBM
`energy` (real, clean)	1.319	**0.802**	0.891	1.709
synthetic (K=3, clean)	**0.585**	0.611	0.943	0.822

quality, and the synthetic control, Huber and quantile beat squared error decisively at essentially every contamination level (winning on all five seeds), with effect sizes of multiples of the target standard deviation—e.g. on energy at 8% contamination Huber attains 2.22 RMSE versus 12.49 for squared error (a 5.6× reduction).

A scale-consistency fix for saturated-gradient objectives. The two large multi-target sets (rf1, scm20d) initially *contradicted* this picture: Huber and quantile *lost* to squared error at low contamination (4–8%), winning only at 16%. The cause is a scale bug, not a robustness failure. Huber's gradient is clipped to $\pm\delta$ and quantile's is $\pm\tau$, so the per-tree update is bounded; with a fixed $\delta = 1$ the effective ratio δ/σ varies by $\sim$25× across these datasets (energy $\approx$0.1, scm20d $\approx$0.004). On large-scale targets (scm20d per-output $\sigma\approx$250, rf1 $\approx$60) the bounded update cannot traverse the target range within $n_{\text{estimators}}$ trees, so the model severely under-fits even on *clean* data—on rf1 the clean Huber RMSE is 21.1 versus 1.5 for squared error ($\sim$15×). The fix makes the objective scale-consistent: we standardize the target per output by its median and $1.4826 \cdot \text{MAD}$ before boosting, fit in the standardized space, and un-standardize predictions, so $\delta = 1$ corresponds to one robust-σ on every target. This removes the clean-fit penalty (rf1 15.0 × →0.9×, scm20d 2.4 × →1.0×) and restores robustness on rf1 and scm20d under contamination (rf1 at 16%: squared 27.6 vs. standardized quantile 2.94; scm20d at 16%: squared 285 vs. standardized Huber 135), turning robust multi-output regression from a dataset-dependent into a *general* win. The standardization is applied automatically for the Huber and quantile objectives. The one honest trade-off is on small-scale targets (energy, $\sigma\approx$10), where the un-standardized objective's accidentally-aggressive clipping had been *more* robust at heavy contamination; standardization gives up a little peak robustness there (Huber at 16%: 2.51→5.68) while still beating squared error (18.76), and a sub-unit δ/σ ratio could recover it.

9.6 Systems: backends

The split search and leaf fit run behind a backend interface with three implementations. The optional Rust backend gives $\sim$5.8× faster constant-leaf training than the NumPy reference while staying *bitwise* parity-tested. An optional CUDA backend (CuPy) accelerates wide and multi-output histograms; on an NVIDIA T4 it ranges from no gain on narrow problems (launch-bound) to 5.5× on wide multi-output (Table 6), and is parity-tested at `allclose` (GPU atomic-add reordering precludes bitwise equality).

9.7 Further ablations

Three results are reported as null/negative, which we consider part of an honest account. **Growth policy:** oblivious (`symmetric`) trees win 10/12 synthetic cells, but the synthetic generators plant exactly the low-order globally-shared thresholds that oblivious trees represent best, and we have no real-data evidence; the default therefore stays `leafwise`. **Adaptive leaf selection:** a per-leaf

Table 6: CUDA vs. NumPy fit time (seconds, 30 trees, NVIDIA T4). Speedup = NumPy/CUDA. GPU pays off on wide and multi-output histograms; narrow problems are launch-bound.

Task (features)	NumPy	CUDA	speedup
binary (30)	2.34	2.44	0.96×
binary (200)	11.18	6.36	1.76×
regression (200)	11.51	7.03	1.64×
multiclass, C=5 (200)	43.70	21.67	2.02×
multi-output, K=5 (200)	52.79	9.59	**5.50**×

leave-one-out gate between constant and embedded-linear leaves [7] tracks the better of the two but never separates from it by more than one standard deviation—a robust opt-in hedge, not a general improvement (it is nonetheless the best-ranked RepLeafGBM arm on the OpenML classification suite, Table 3). **Binary classification:** no reweighting, Hessian-floor, or ℓ_2 remedy makes embedded-linear leaves beat constant leaves on binary tasks (consistent across 16 configuration–dataset cells); we trace this to the logistic Hessian $h = p(1-p)$ shrinking the leaf-linear signal, and recommend `constant` leaves for binary problems.

10 Discussion

Two complementary channels. RepLeafGBM separates *where to be local* from *how to predict locally*. Routing (I1) supplies axis-aligned, categorical-native, interpretable locality at GBDT cost; the leaf model supplies smooth, non-linear response *within* a region via a learned basis. A constant-leaf GBDT can only approximate a smooth effect (e.g. a photometric color or a redshift trend) by many small steps; an embedded-linear leaf captures it directly through $w^\top z$. Because explicit engineered features enter the leaf's linear model, arithmetic features can help even though the *splits* are scale-invariant — a property absent from constant-leaf GBDTs.

Positioning. The routing half is the histogram GBDT of LightGBM/XGBoost/CatBoost [3, 13, 19]. The leaf half borrows from tabular deep learning, where piecewise-linear and periodic numeric *embeddings* improve neural tabular models [9]; RepLeafGBM consumes such an embedding as a frozen leaf basis rather than as the input to a deep network, keeping the tree explicit. Oblivious (`symmetric`) trees connect it to CatBoost-style regularized routing [19].

RepLeafGBM as an ensemble member. Because RepLeafGBM is architecturally distinct from a standard GBDT, a natural question is whether it adds value inside an ensemble of tuned models. Averaging it into a tuned-GBDT blend never hurts, and on numerical regression it yields a sign-consistent improvement (Wilcoxon $p = 0.0446$, winning or tying on all 19 datasets); but the effect is practically negligible (only one dataset clears the relevance band, and the bootstrap mean-gain interval straddles zero), and its prediction-error decorrelation from the GBDTs is marginal ($\Delta r \approx 0.002$) because it still routes on raw features. The robustly-supported finding is the generic one—ensembling beats the validation-selected best single model on most datasets—rather than a RepLeafGBM-specific diversity benefit. RepLeafGBM is thus a distinct blend member that never degrades an ensemble, with a diversity premium that is real in sign but small in magnitude.

When the leaf channel helps. The experiments draw a consistent line. The embedded-linear leaf earns its keep when a region carries a *smooth* effect that a constant cannot capture but a low-dimensional affine model on $z_\theta(x)$ can: real regression with the extrapolation guard (§9.2), an external GBDT's own routes refit with embedded leaves (§9.1), and—most decisively—robust multi-output regression under target contamination (§9.5). It does *not* help binary classification: the logistic Hessian $h = p(1-p) \to 0$ in confident regions starves the h-weighted ridge fit, so the leaf-linear weights chase noise and constant leaves are both more accurate and cheaper. This dichotomy—linear leaves pay off exactly when the Newton targets carry recoverable smooth structure—is the paper's main practical takeaway.

Interpretability and the frozen-encoder trade-off. Because routing is a raw-feature decision tree, the partition is exactly as inspectable as a LightGBM/XGBoost model; the representation enters only the within-leaf prediction, so the "which leaf and why" explanation is unchanged. Freezing the encoder (Proposition 1) is what keeps each leaf solve a convex, closed-form ridge regression and the boosting loop a coordinate descent in function space. The price is that the representation cannot co-adapt with the ensemble; lifting it would require alternating optimization with re-evaluation of earlier leaves and prediction-cache invalidation, which we leave to future work (§11).

11 Limitations and future work

Method. (i) The Newton-target leaf fit is a second-order approximation for non-squared losses; no line search or leaf-wise damping beyond λ is performed. (ii) The encoder is frozen: the alternating optimization that would let θ co-adapt with the ensemble (§6) is future work. (iii) If the encoder's output dimension exceeds a cap `max_leaf_emb_dim`, a random projection is applied, which preserves structure only in expectation and can dilute informative dimensions. (iv) Histogram bin edges are computed once per fit (no per-node re-quantization), and native training routes missing values left at every split rather than learning a default direction.

Evaluation. (v) Our baselines are GBDTs and scikit-learn; we have not yet benchmarked against neural tabular models such as TabNet [2], NODE [18], or FT-Transformer [8], so the positioning relative to tabular deep learning is argued rather than measured. (vi) The fair leaderboard uses five seeds and fifty HPO trials per model; the niche studies (robust multi-output, router extraction) use five seeds, at which the two-sided Wilcoxon test floors at $p = 0.0625$—their effect directions are consistent across seeds, but formal significance awaits a seven-to-ten-seed rerun, and the smaller synthetic and guard studies should be read as indicative. (vii) The per-output target standardization that makes robust objectives scale-consistent gives up a little peak robustness on small-scale targets; a tuned sub-unit δ/σ ratio is left to future work. (viii) Two empirical questions are open: why joint vector-leaf routing trails per-output CatBoost/XGBoost on *clean* real multi-output data, and whether the oblivious-tree advantage survives on real data under a fair capacity match. (ix) High-cardinality categorical features still trail LightGBM's native categorical pipeline, and the leaderboard's symmetric ordinal encoding leaves RepLeafGBM's native categorical splits untested.

12 Conclusion

RepLeafGBM keeps the raw-feature, axis-aligned routing that makes GBDTs strong on tabular data and adds expressivity only inside the leaf, as an h-weighted ridge fit of an affine model on a learned, frozen representation. Freezing the encoder is not a convenience but a correctness

requirement: updating it mid-boosting breaks the stage-wise additive structure of gradient boosting (Proposition 1). Empirically, under a shared HPO budget on the Grinsztajn suites RepLeafGBM is a competitive *mid-pack* learner rather than a state-of-the-art one; its distinctive value is in niche regimes—the representation-conditioned leaf is a real, separable contribution that improves an external GBDT's own routes, and joint vector-leaf routing with scale-consistent robust objectives is a decisive, general win under target contamination. We report candidly where the idea does not help, namely binary classification and the transfer of *learned* encoders to real tabular targets. We hope the explicit separation of *where to be local* (raw routing) from *how to predict locally* (a representation-conditioned leaf), together with an honest map of when each channel pays off, is a useful point in the design space between gradient-boosted trees and tabular deep learning.

Acknowledgments

The RepLeafGBM library and this manuscript were developed with extensive use of Claude Code (Anthropic) as an AI coding and research assistant—for implementation, architectural design, running and analyzing the experiments, and drafting the paper. All design decisions, experimental verification, and the final content are the author's responsibility.

A Objectives

Each objective supplies a per-row gradient g_i and Hessian h_i (used in the split gain and as the leaf-fit weights), an optimal constant initial score F_0, and an output transform ϕ (Table 7). All leaves are then fitted by h-weighted least squares on the Newton targets $t_i = -g_i/h_i$ (§3). Here $p = \sigma(F)$ for binary, $p_k = \mathrm{softmax}(F)_k$ for multiclass, and $\mu = e^F$ for Poisson; for the quantile loss $g_i = 1 - \alpha$ where $F \geq y$ and $-\alpha$ otherwise. Multiclass grows one tree per class per round; multi-output grows one shared tree per round with vector leaves (constant-Hessian losses only).

Table 7: Objective definitions implemented by RepLeafGBM.

Objective	g_i	h_i	F_0	ϕ
Regression (squared)	$F - y$	1	$\overline{y}$	identity
Binary (logistic)	$p - y$	$p(1-p)$	$\log \frac{\bar{p}}{1-\bar{p}}$	σ
Huber	$\mathrm{clip}(F - y, \pm\delta)$	1	$\mathrm{median}(y)$	identity
Quantile (α)	$(1-\alpha)\,/\,-\alpha$	1	$Q_\alpha(y)$	identity
Poisson	$\mu - y$	μ	$\log \overline{y}$	exp
Multiclass (softmax)	$p_k - \mathbf{1}[y = k]$	$p_k(1-p_k)$	$\log \mathrm{prior}_k$	softmax
Multi-output (squared)	$F - Y$	1	$\overline{Y}_{\cdot k}$	identity

B Reproducibility and additional results

Manifest. The OpenML suite (Table 3) uses seeds $\{0, 1, 2\}$, a 60/20/20 split (stratified for classification), early stopping after 25 rounds, and at most 6,000 rows per dataset; datasets are anchored by OpenML `data_id`: `house_sales` (42731), `diamonds` (42225), `wine_quality` (287), `credit_g` (31), `phoneme` (1489), `adult` (1590), `wine` (187), `vehicle` (54), plus the scikit-learn `california` set; multi-output uses energy-efficiency (1472). Package versions: Python 3.11, numpy 1.23.5, scikit-learn 1.2.0, lightgbm 4.6.0, xgboost 3.2.0, catboost 1.2.10, torch 2.9.1. The NumPy

and Rust backends are parity-tested *bitwise* (identical histograms, leaf fits, and predictions); the CUDA backend is parity-tested at `allclose` because GPU atomic-add reordering precludes bitwise equality. The full per-dataset tables and the harness that produced every number are included with the open-source release.

Per-dataset OpenML results. Table 8 gives the per-dataset primary metric behind the mean ranks of Table 3, for a representative subset of models. Tuned GBDTs lead the regression and binary datasets, while RepLeafGBM with embedded-linear leaves wins both multiclass datasets among the models shown (`wine` outright across all models).

Table 8: Per-dataset OpenML primary metric (↓; RMSE for regression, log-loss for classification; 3 seeds). RL = RepLeafGBM (constant / embedded-linear / adaptive leaves, `identity` encoder). Bold = best among the models shown.

Dataset	CatBoost	LightGBM	XGBoost	RL-const	RL-emb	RL-adapt
Regression (RMSE)						
`california`	**0.5013**	0.5119	0.5139	0.5110	0.5080	0.5109
`house_sales`	**0.1654**	0.1712	0.1725	0.1719	0.1775	0.1721
`diamonds`	0.1064	**0.1035**	0.1058	0.1095	0.1114	0.1108
`wine_quality`	0.6542	**0.6479**	0.6499	0.6521	0.6505	0.6514
Binary (log-loss)						
`credit_g`	**0.5126**	0.5365	0.5304	0.5491	0.5419	0.5405
`phoneme`	**0.2584**	0.2668	0.2756	0.2697	0.2680	0.2659
`adult`	0.3154	0.3167	**0.3144**	0.3235	0.3281	0.3239
Multiclass (log-loss)						
`wine`	0.1258	0.1629	0.1428	0.1083	**0.0947**	0.1013
`vehicle`	0.5025	0.4995	0.5313	0.5194	**0.4985**	0.5053

Synthetic leaderboard. Table 9 reports an indicative large-synthetic benchmark (n=10,000, 20 features; single configuration, not multi-seed-averaged), showing RepLeafGBM is competitive with the tuned GBDTs on synthetic regression and binary tasks.

Table 9: Indicative large-synthetic benchmark (n=10k, 20 features). Regression RMSE and binary log-loss (↓). Single configuration; not a multi-seed leaderboard.

Model	Regression RMSE	Binary log-loss
CatBoost	**0.390**	**0.256**
HistGradientBoosting	0.407	0.258
LightGBM	0.413	0.258
RepLeafGBM (constant)	0.405	0.260
RepLeafGBM (embedded-linear)	0.400	0.279

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM*

SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pages 2623–2631, 2019.

[2] Sercan Ö. Arik and Tomas Pfister. TabNet: Attentive interpretable tabular learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6679–6687, 2021.

[3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016.

[4] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[5] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Additive logistic regression: A statistical view of boosting. *The Annals of Statistics*, 28(2):337–407, 2000.

[6] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[7] Gene H. Golub, Michael Heath, and Grace Wahba. Generalized cross-validation as a method for choosing a good ridge parameter. *Technometrics*, 21(2):215–223, 1979.

[8] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[9] Yury Gorishniy, Ivan Rubachev, and Artem Babenko. On embeddings for numerical features in tabular deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[10] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[11] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[12] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[14] Guolin Ke, Zhenhui Xu, Jia Zhang, Jiang Bian, and Tie-Yan Liu. DeepGBM: A deep learning framework distilled by GBDT for online prediction tasks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 384–394, 2019.

[15] Peter Kontschieder, Madalina Fiterau, Antonio Criminisi, and Samuel Rota Bulò. Deep neural decision forests. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1467–1475, 2015.

[16] Niels Landwehr, Mark Hall, and Eibe Frank. Logistic model trees. *Machine Learning*, 59(1–2): 161–205, 2005.

[17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[18] Sergei Popov, Stanislav Morozov, and Artem Babenko. Neural oblivious decision ensembles for deep learning on tabular data. In *International Conference on Learning Representations (ICLR)*, 2020.

[19] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[20] J. Ross Quinlan. Learning with continuous classes. In *Proceedings of the 5th Australian Joint Conference on Artificial Intelligence*, pages 343–348, 1992.

[21] Yu Shi, Jian Li, and Zhize Li. Gradient boosting with piece-wise linear regression trees. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3432–3438, 2019.

[22] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014.

[23] Yong Wang and Ian H. Witten. Inducing model trees for continuous classes. In *Proceedings of the 9th European Conference on Machine Learning (Poster Papers)*, pages 128–137, 1997.